Unit 1 – Using Objects and Methods Name: **KEY**
Checkpoint Practice 2

1) Match the word with definition. You will use each vocab word only once.

A. return	B. overloading	C. instance variables	D. object	E. dot
F. parameters	G. behavior	H. class	I. default	J. methods
K. new	L. constructor			

H a) defines a new data type. It is the formal implementation, or blueprint, of the attributes and behaviors of the objects of that class.

D b) a specific instance of a class with defined attributes.

L c) code that is used to create new objects and initialize the object's attributes.

K d) keyword used to create objects with a call to one of the class's constructors.

C e) define the attributes for objects.

J f) define the behaviors or functions for objects.

E g) operator - used to access an object's methods.

F h) the values or data passed to an object's method inside the parentheses in the method call to help the method do its job.

A i) keyword used to return values to the calling method.

B j) when there is more than one constructor; they must differ in the number, type, or order of parameters.

I k) constructor doesn't take any arguments.

G l) what methods define for all objects of the class.

2) What is the relationship between classes and objects?

A class is a blueprint to create an object and an object is an instance of a class. The class specifics how instances are created and how they behave.

3) What is the purpose of a constructor in a class?

The constructor is called when an object of a class is created and it gives the object's instance variables values.

4) Fill in the blanks with the appropriate code that would go in this object creating class.

```
public class Turtle {
     /* create three instance variables
     *  age (int), name (String), weight (double)
     */

     private int age;
     private String name;
     private double weight;



     /* create a default constructor that sets age to 0,
     *  name to "Unknown", and weight to 0
     */

     public Turtle() {
          age = 0;
          name = "Unknown";
          weight = 0;
     }



     // create a parameter constructor

     public Turtle(int a, String b, double c) {
          age = a;
          name = b;
          weight = c;
     }


     //create a method called "eat" that causes weight to increase by 1.5

     public void eat() {
          weight = weight + 1.5;
     }



     /* create a method called "speak" that
     *  displays the turtle's data
     */

     public void speak() {
          System.out.println("Age: " + age);
          System.out.println("Name: " + name);
          System.out.println("Weight: " + weight);
     }
```

```
	/* create a method called "walk" that causes the
	*  weight to decrease by 0.5
	*/

	public void walk() {
		weight = weight - 0.5;
	}


}
```

Use the following code to answer questions 5 – 7:

```
public class BabyRecord
{
	private String name;
	private int age;
	private int weight;
	private boolean needsVaccine;

	public BabyRecord(String n, int a, int w, boolean v)
	{
		name = n;
		age = a;
		weight = w;
		needsVaccine = v;
	}

	public BabyRecord(String n, int a, int w)
	{
		name = n;
		age = a;
		weight = w;
		needsVaccine = true;
	}
}
```

__A__ 5) What happens when the following statement is executed?

```
BabyRecord b = new BabyRecord("Charlie", 1, 20);
```

(A) It calls the second constructor, setting `needsVaccine` to `true`.
(B) It calls the first constructor, setting `needsVaccine` to `false`.
(C) It calls the default constructor because no matching constructor exists.
(D) It results in a compilation error because no matching constructor exists.

D 6) A new constructor is to be added to the `BabyRecord` class. Which of the following is NOT a possible header for the new constructor?

(A) `BabyRecord(int a, int w)`
(B) `BabyRecord(String n, int a)`
(C) `BabyRecord(String n, boolean v)`
(D) `BabyRecord(String n, int w, int a)`

A 7) Which of the following statements correctly creates a `BabyRecord` object using the first constructor?

(A) `BabyRecord b = new BabyRecord("Alice", 2, 25, true);`
(B) `BabyRecord b = new BabyRecord("Alice", 2, 25);`
(C) `BabyRecord b = new BabyRecord("Alice", true, 2, 25);`
(D) `BabyRecord b = new BabyRecord();`

8) Consider the following class.

```
public class Liquid
{
     private double boilingPoint;
     private double freezingPoint;
     private double currentTemp;

     public Liquid(double b, double f, double c)
     {
          boilingPoint = b;
          freezingPoint = f;
          currentTemp = c;
     }

     public void lowerTemp(double amount) {
          currentTemp -= amount;
     }

     public void freeze() {
          currentTemp = freezingPoint;
     }
}
```

Assume that the statement appears in a method in a class other than `Liquid`:

```
Liquid water = new Liquid(100, 0, 30);
```

There is only ONE valid statement below. Circle it.

```
water.freeze;          water.lowerTemp();       Liquid.lowerTemp(9);

water.freeze(30);      water.freeze();          Liquid.freeze();
```

B 9) A student has created an `Animal` class. The class contains variables to represent the following:

- A String variable called `species` that represents the species of the `Animal`
- An `int` variable called `age` that represents the age of the `Animal`
- A String variable called `gender` that represents the gender of the `Animal`
- The object `elephant` will be declared as type `Animal`

Which of the following descriptions is accurate?

(A) An instance of the `elephant` class is `Animal`.
(B) An instance of the `Animal` class is `elephant`.
(C) An attribute of the `age` object is `int`.
(D) An attribute of the `Animal` instance is `species`.

D 10) Consider the following class.

```
public class Flower
{
     public Flower(int stems)
     { /* implementation not shown*/ }

     public Flower(int stems, int leaves)
     { /* implementation not shown*/ }

     public Flower(int stems, String name)
     { /* implementation not shown*/ }
}
```

Which of the following constructors can be added to the `Flower` class without causing a compile-time error?

I
```
     public Flower(int branches)
     { /* implementation not shown*/ }
```

II
```
     public Flower(String name, int stems)
     { /* implementation not shown*/ }
```

III
```
     public Flower(int stems, int leaves, int branches)
     { /* implementation not shown*/ }
```

(A) I only
(B) II only
(C) I and III only
(D) II and III only

11) Consider the following line of code.

```
double cost = 10.95;
```

Circle/highlight the statements that correctly round the variable `cost` to 11.

I.
```
cost = (int) cost;
```

II.
```
cost = (int)(cost - 0.5);
```

III.
```
cost = (int)(cost + 0.5);
```

IV.
```
cost = (int)cost + 0.5;
```

I will truncate to 10

II will truncate to 10

IV will truncate to 10 then add 0.5

12) Circle/highlight the pairs of statements that will cause an error message.

I.
```
double x = 10.7;
int y = x;
```

II.
```
double x = 10.7;
int y = (int) x;
```

III.
```
int x = 10;
double y = x;
```

I is the only one that will cause an error because you are trying to fit a double into an int. II correctly casts to fit and III stores an int in a double, which is allowed.